Bahdanau Attention and Luong Attention

Contents

1. Overview 2
2. Prerequisites 3
3. Recap: what both must compute 3
4. Bahdanau attention 4
    4.1 What the score depends on 4
    4.2 The alignment network, step by step 5
5. Luong attention 7
    5.1 A simpler score 7
    5.2 The current state, and the context on the output side 9
6. The two compared 10
    6.1 Summary of the differences 10
    6.2 The experiment 11
7. Summary 14
8. Sources 14
9. Key terms 15

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Attention mechanism.
- **Compare with:** Cross-attention.

1. Overview

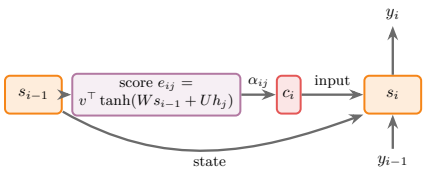
Key point: Both attentions build the context vector as a weighted mix of the encoder states. **Bahdanau** scores with a small neural network and feeds the context *into* the decoder's LSTM step; **Luong** scores with a **dot product** (G-634) and joins the context to the LSTM's *output*.

In more detail, the two differ in two things:

- **How each encoder state is scored.** Bahdanau (additive) attention scores the previous decoder state s_{i-1} against each encoder state h_j with a small neural network. Luong (multiplicative) attention scores the current state s_i against h_j with a dot product.
- **Where the context vector c_i enters the decoder.** Bahdanau feeds c_i into the LSTM step; Luong joins c_i to the LSTM's output.

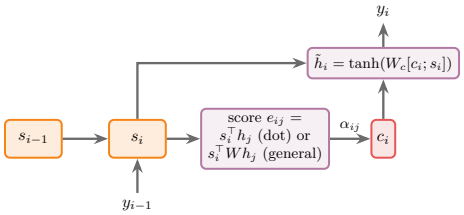
Attention (G-226) in the encoder–decoder works like this: at every decoder step, the **context vector** (G-460; a list of numbers) is a weighted sum of all the encoder's hidden states. Here h_j is the hidden state (a vector) that the encoder produced for input word j, s_i is the decoder's hidden state at output step i, and c_i is the context vector for that step. The weights are the **attention weights** (G-224) α_{ij}, and they come from a **softmax** (G-1830; it turns raw scores into positive weights that add up to 1, see the softmax function) over raw scores, the **alignment scores** (G-190) e_{ij}. The attention section left open how the scores are computed. This Note opens the two best-known answers: Bahdanau et al. (2015) and Luong et al. (2015). Luong's dot-product score is, apart from a scaling factor, the one the transformer uses (Vaswani et al. 2017, section 3.2.1).

(a) Bahdanau: attention *before* the LSTM step



c_i is an *input* of the LSTM step, so step i cannot start before c_i is known

(b) Luong: attention *after* the LSTM step



c_i joins the *output* side; the LSTM itself never needs it

Figure 1: Where attention sits in the two designs. (a) Bahdanau: the previous state s_{i-1} is scored, and c_i is an input of the LSTM step. (b) Luong: the LSTM step runs first, its new state s_i is scored, and c_i is combined with s_i before the output layer

2. Prerequisites

- **Attention** (G-226): context vectors, **alignment scores** (G-190), softmax over the input positions.
- Encoder–decoder: encoder, decoder, teacher forcing.
- Dot product and cosine similarity: the **dot product** (G-634) as a measure of similarity.
- **Forward propagation** (G-797): a dense layer as a matrix product.

3. Recap: what both must compute

> **Key point:** Both designs do the same three things at every decoder step: score every encoder state (e_{ij}), turn the scores into weights with a softmax (α_{ij}), and add up the weighted states into the context $c_i = \sum_j \alpha_{ij} h_j$. Only the score function and the wiring differ.

For "turn off the lights" → "light band karo", decoder step 1 needs a context vector c_1, a weighted sum of the four encoder states:

$$c_1 = \alpha_{11} h_1 + \alpha_{12} h_2$$

$$+ \alpha_{13} h_3 + \alpha_{14} h_4$$

Step 2 needs c_2 with new weights, and so on: (number of input words) $\times$ (number of output words) weights in all. Each weight is a word-to-word similarity: α_{11} says how much "turn" counts when writing "light". The question of this Note is how to get the raw scores e_{ij}: the choice of **score function** (G-1752).

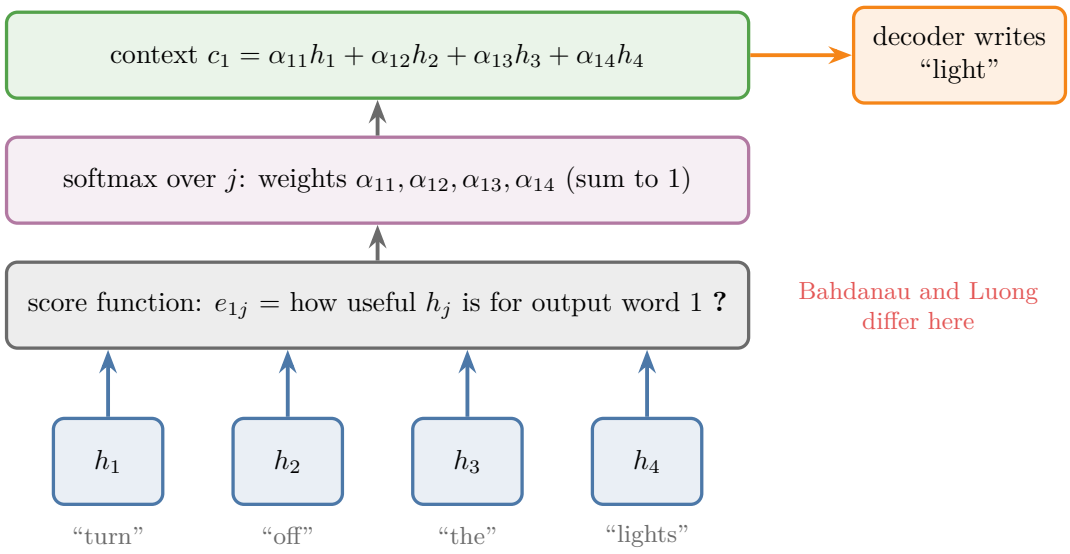


Figure 2: The three stages that both attentions share, for the first output word of "turn off the lights": score every encoder state, turn the scores into weights with a softmax, and add up the weighted states into the context vector c_1. Only the score function, the grey box, differs between Bahdanau and Luong

In Figure 2, the softmax and the weighted sum are fixed; everything this Note compares happens inside the grey box.

4. Bahdanau attention

> **Key point:** A small neural network with one hidden layer looks at the decoder's previous state and one encoder state, and gives the pair a score: $e_{ij} = v^\top \tanh(W[s_{i-1}; h_j])$. The context vector is then an input of decoder step i.

4.1 What the score depends on

> **Key point:** On the encoder state h_j and on the decoder's *previous* state s_{i-1}, which holds what has been translated so far.

As what the weight depends on explains, α_{ij} must depend on h_j, the input word being judged, and on what the decoder has already written, which is stored in s_{i-1}. To compute α_{11}, the weight of "turn" for the first output word, we need h_1 and s_0; for α_{21} we need h_1 and s_1. Bahdanau et al. (2015) do not choose a formula for the score; they let a small **feed-forward**

network (G-775; a network with no loops, here one hidden layer) learn it, the **alignment model** (G-189).

4.2 The alignment network, step by step

> **Key point:** Concatenate s_{i-1} with every h_j, pass the rows through a hidden layer with tanh, then through one output unit: one score per input word. Softmax gives the weights.

Take hidden states of size 4 and an alignment network with 3 hidden units and 1 output unit. At decoder step 1:

1. **Concatenate.** Join s_0 (a vector of 4 numbers) with each of $h_1, \ldots, h_4$ (4 numbers each). Stacked, the four rows form a 4×8 matrix X, one row per input word.
2. **Hidden layer.** Multiply by the 8×3 weight matrix W and apply tanh: a 4×3 matrix. A bias can be added before the tanh.
3. **Output unit.** Multiply by the 3×1 vector v: this gives a vector of 4 scores $e_{11}, \ldots, e_{14}$, one per input word.
4. **Softmax** over the 4 scores gives $\alpha_{11}, \ldots, \alpha_{14}$, and the weighted sum gives c_1.
5. **Decoder step.** c_1, s_0 and the input y_0 (`<start>`) go into the LSTM, which outputs the first word, "light", and the new state s_1.

At step 2 the same network runs again with s_1 in place of s_0. In the 4×8 matrix only the left half changes; the encoder states on the right stay the same. The changing decoder state is why every step gets different weights. The network's weights are shared across all decoder steps, like a time-distributed dense layer (one dense layer applied with the same weights at every step), and are updated by backpropagation together with the two LSTMs.

1. **In words:** concatenate the two states, apply one tanh hidden layer, then one output unit; softmax over the input positions.
2. **Formula** (Bahdanau et al. 2015, appendix A.1.2):

$$e_{ij} = v^\top \tanh\left(W[s_{i-1}; h_j]\right)$$

$$e_{ij} = v^\top \tanh\left(W_a s_{i-1} + U_a h_j\right)$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}$$

$$c_i = \sum_j \alpha_{ij} h_j$$

Here $[s; h]$ is s and h joined into one long vector, W and v are learned weights, and $v^\top$ is v written as a row, so $v^\top x$ is the sum of products of matching entries. The two forms are the same: multiplying the joined vector $[s; h]$ by W equals multiplying s by the left half of W (called W_a) and h by the right half (called U_a), and adding.

3. **Example:** the decoder state $s = [0.5, -0.2, 0.8, 0.1]$ and four encoder states, the first being $h_1 = [0.3, -0.5, -0.9, -1.0]$ (the Notebook lists all four and the weights W and $v = [0.1, -0.4, 0.2]$). For $j = 1$, the joined row is $[0.5, -0.2, 0.8, 0.1, 0.3, -0.5, -0.9, -1.0]$.

   Multiplying the joined row by W gives a vector of 3 numbers, one per hidden unit. Each number is the sum of 8 products: an input times its weight in that unit's column of W. Every product, then the column sums:

Input	W, unit 1	W, unit 2	W, unit 3	Product, unit 1	Product, unit 2	Product, unit 3
0.5	0.7	0.1	-0.4	0.35	0.05	-0.20
-0.2	-0.2	-0.9	-0.8	0.04	0.18	0.16
0.8	0.3	0.3	0.2	0.24	0.24	0.16
0.1	-0.2	1.0	1.0	-0.02	0.10	0.10
0.3	0.4	0.3	0.4	0.12	0.09	0.12
-0.5	-0.2	-0.7	0.4	0.10	0.35	-0.20
-0.9	0.1	-0.4	0.0	-0.09	0.36	0.00
-1.0	0.8	0.9	-0.3	-0.80	-0.90	0.30
Sum				-0.06	0.47	0.44

   The three sums are $[-0.06, 0.47, 0.44]$. The tanh of each gives $[-0.060, 0.438, 0.414]$. Then each is multiplied by its weight in v and the results are added:

$$0.1 \times (-0.060) = -0.0060$$

$$-0.4 \times 0.438 = -0.1752$$

$$0.2 \times 0.414 = 0.0828$$

$$e_1 = -0.0060 - 0.1752 + 0.0828 = -0.0984$$

   The Notebook, using unrounded values, gives $e_1 = -0.099$.

   The four scores are $(-0.099, -0.011, 0.449, -0.116)$, and softmax turns them into the weights $(0.208, 0.227, 0.360, 0.205)$. The **score function** (G-1752) has 27 learned numbers:

$$8 \times 3 + 3 = 27$$

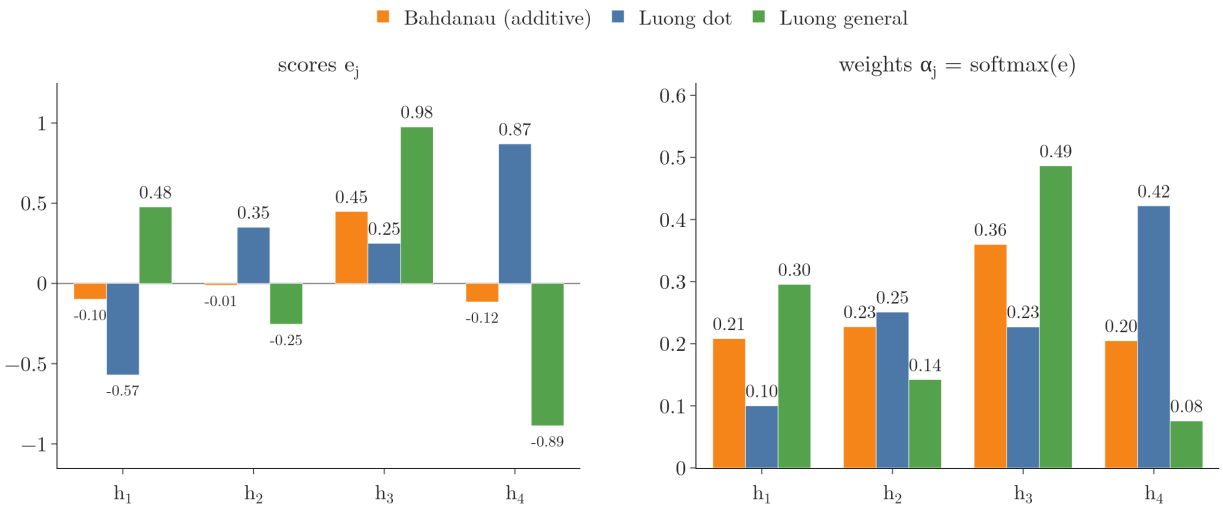


Figure 3: The same decoder state and four encoder states scored three ways: Bahdanau's additive network (this section) and Luong's dot and general scores (section 5.1). Left: the raw scores. Right: the weights after the softmax

Figure 3 puts this example next to the two Luong scores of section 5.1. Each function ranks the four states differently: the additive network favours h_3, the dot score h_4, the general score h_3 again but more strongly. The softmax keeps the order of the scores and turns them into weights that add up to 1.

Because the score adds a term from s_{i-1} to a term from h_j inside the tanh, Bahdanau attention is also called **additive attention** (G-255; Vaswani et al. 2017, section 3.2.1). Since $U_a h_j$ does not depend on i, it can be computed once per sentence (Bahdanau et al. 2015, appendix A.1.2).

5. Luong attention

> **Key point:** Two changes. The score uses the *current* decoder state s_i, and it is a simple product: $e_{ij} = s_i^\top h_j$ (dot) or $s_i^\top W_a h_j$ (general). The context vector is joined to s_i after the LSTM step, not fed into it.

Luong, Pham and Manning (2015) kept the goal and changed the means.

5.1 A simpler score

> **Key point:** Two similar vectors have a large dot product. Using the dot product as the score needs no network at all.

The aim of the score is not to approximate some exact function; it is to find which encoder states are useful now. A similarity measure does that job, and the simplest one is the **dot product** (G-634) (the sum of products of matching entries, see the geometric meaning of the

dot product): large when two vectors point the same way, small or negative when they do not. Luong et al. (2015, section 3.1) proposed three content-based scores:

Name	Score e_{ij}	Learned parameters
dot	$s_i^\top h_j$	none
general	$s_i^\top W_a h_j$	one matrix W_a
concat	$v_a^\top \tanh(W_a[s_i; h_j])$	W_a and v_a, as in Bahdanau

The dot score requires s_i and h_j to have the same size; the general score lifts that requirement and lets the model learn which directions of similarity matter (SLP3 §14.8). Because the score multiplies the two states, this family is called **multiplicative attention** (G-1137; Vaswani et al. 2017, section 3.2.1).

1. **In words:** multiply the decoder state and each encoder state element by element and add up (dot), or first transform the encoder state by a learned matrix (general).
2. **Formula:**
$$e_{ij} = s_i^\top h_j \quad \text{(dot)}$$
$$e_{ij} = s_i^\top W_a h_j \quad \text{(general)}$$
3. **Example:** the same $s = [0.5, -0.2, 0.8, 0.1]$ and $h_1 = [0.3, -0.5, -0.9, -1.0]$:
$$e_1 = 0.5(0.3) + (-0.2)(-0.5)$$
$$+ 0.8(-0.9) + 0.1(-1.0)$$
$$e_1 = 0.15 + 0.10 - 0.72 - 0.10$$
$$e_1 = -0.57$$
The four dot scores are $(-0.57, 0.35, 0.25, 0.87)$, with weights $(0.100, 0.251, 0.227, 0.422)$: no learned numbers at all. With a 4×4 matrix W_a (16 learned numbers, in the Notebook) the general scores are $(0.477, -0.254, 0.975, -0.887)$, with weights $(0.296, 0.142, 0.486, 0.076)$. The three functions rank the four states differently. Here the states and the weights are random numbers; in a trained model, training sets them so that the useful encoder states get the high scores, whichever function is used.

5.2 The current state, and the context on the output side

> **Key point:** The LSTM step runs first. Its new state s_i is scored against the encoder states, and $\tilde{h}_i = \tanh(W_c[c_i; s_i])$ feeds the softmax.

Bahdanau's decoder goes $s_{i-1} \to \alpha_i \to c_i \to s_i$: the context must be ready before the LSTM step. Luong's goes $s_i \to \alpha_i \to c_i \to \tilde{h}_i$ (Luong et al. 2015, section 3.1):

1. The decoder LSTM takes y_{i-1} and s_{i-1} and produces s_i, with no context vector.
2. s_i is scored against every h_j; softmax gives α_{ij}; the weighted sum gives c_i.
3. c_i and s_i are joined and passed through a dense layer with tanh: the **attentional hidden state** (G-227) $\tilde{h}_i = \tanh(W_c[c_i; s_i])$.
4. A softmax layer on $\tilde{h}_i$ gives the next word: $p(y_i) = \text{softmax}(W_s\tilde{h}_i)$.
5. s_i (not $\tilde{h}_i$) and the word y_i go on to step $i + 1$.

Scoring with s_i uses the most recent state, which already includes the latest word written. Figure 1 shows the two wirings side by side.

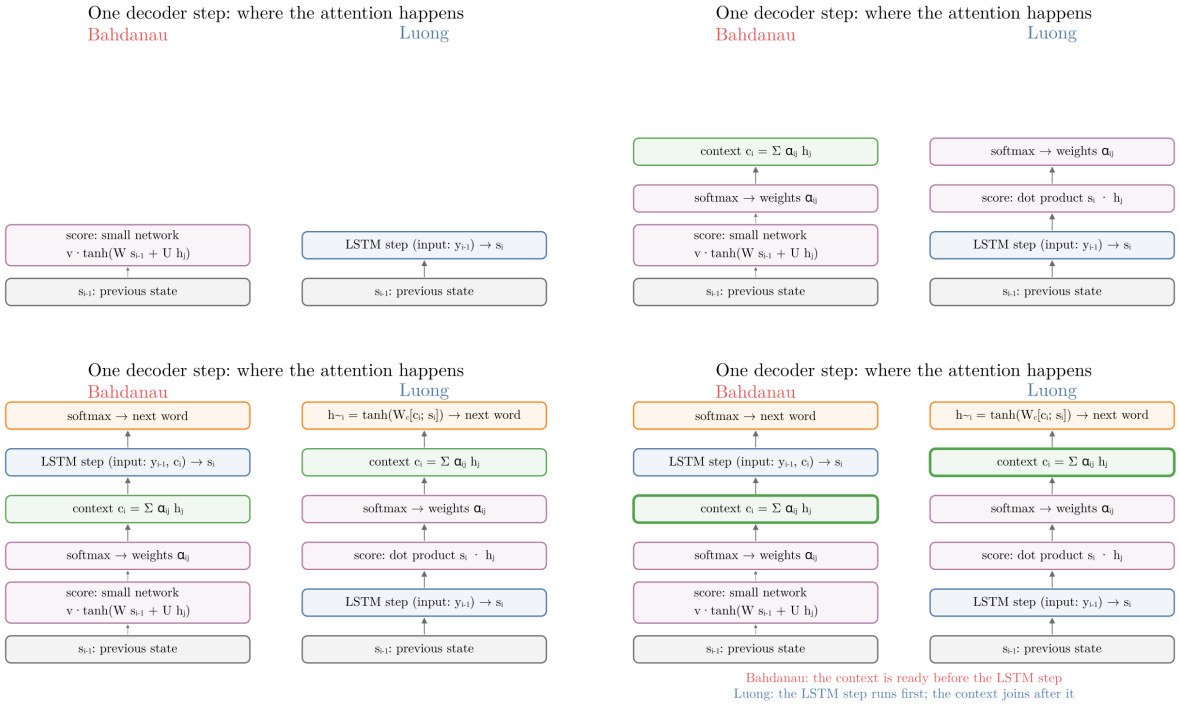


Figure 4: One decoder step in the two designs, built up one operation at a time from the bottom. Bahdanau scores the previous state and computes the context before the LSTM step; Luong runs the LSTM step first, scores the new state by a dot product, and joins the context after the step

Figure 4 builds the same step in both designs, one operation at a time, from the bottom up. Watch where the green context box lands: below the LSTM step on the left, above it on the right.

Python: Luong attention for all decoder steps at once (training, teacher forcing). `S`: decoder states, shape (batch, m, 256); `hs`: encoder states, (batch, n, 256).

```
keys = Wa(hs) if general else hs                    # general: s^T W h = s . (W h)
e = tf.einsum("bid,bjd->bij", S, keys)              # scores, (batch, m, n)
a = tf.nn.softmax(e - 1e9 * pad[:, None, :], axis=-1)  # padding gets weight 0
ctx = tf.einsum("bij,bjd->bid", a, hs)              # context vectors, (batch, m, 256)
logits = out(Wc(tf.concat([ctx, S], -1)))           # Wc: Dense(256, "tanh")
```

Extra: Luong et al. (2015, section 3.3) also propose **input feeding** (G-949): $\tilde{h}_{i-1}$ is joined to the next input, so that the model remembers its past alignment choices. With input feeding the LSTM again needs the previous step's **attention** (G-226) before it can run. The Notebook leaves it out, as in Luong's basic global model of section 3.1.

6. The two compared

Key point: On the same data, Luong's dot **attention** (G-226) translated best: test BLEU 31.6 against 25.1 for Bahdanau's additive attention, with a third fewer parameters and less than half the time per epoch. Luong's general score did worst (17.7): its model learned to put all its weight on the final full stop.

6.1 Summary of the differences

Key point: Which decoder state, which score, where the context goes.

	Bahdanau (2015)	Luong (2015)
Decoder state scored	previous, s_{i-1}	current, s_i
Score e_{ij}	$v^\top \tanh(W_a s_{i-1} + U_a h_j)$	$s_i^\top h_j$ (dot) or $s_i^\top W_a h_j$ (general)
Other name	additive attention	multiplicative attention
Learned parameters in the score	W_a, U_a, v	none (dot) or W_a (general)
Where c_i goes	input of the LSTM step	joined to s_i after the step: $\tilde{h}_i = \tanh(W_c[c_i; s_i])$

Luong et al. (2015, section 3.1) describe their path $s_i \to \alpha_i \to c_i \to \tilde{h}_i$ as "simpler" than Bahdanau's. Vaswani et al. (2017, section 3.2.1) add that the two are similar in theoretical complexity, but dot-product **attention** (G-226) "is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code".

6.2 The experiment

> **Key point:** Three **attention** (G-226) models on the same 60,000 sentence pairs, 3 runs each.

The Notebook trains the Bahdanau attention model of letting a neural network find the function and two Luong models (dot and general) on the same English–French data, with the same bidirectional encoder (it reads the sentence in both directions) and the same decoder LSTM of 256 units, for 12 epochs, 3 runs each. Translation quality is measured by the **BLEU score** (G-315): how many word sequences of a translation match a human reference. The Luong models follow equations 5–7 of Luong et al. (2015) without **input feeding** (G-949).

	Bahdanau (concat)	Luong dot	Luong general
Parameters	8.42 million	5.66 million	5.73 million
Seconds per epoch (mean of 3 runs, shared GPU)	51	22	20
Final training loss	0.77	0.80	1.06
Test BLEU, all 1,000 sentences	25.1	**31.6**	17.7
Test BLEU, sentences of 11–16 words	20.0	**26.8**	12.0

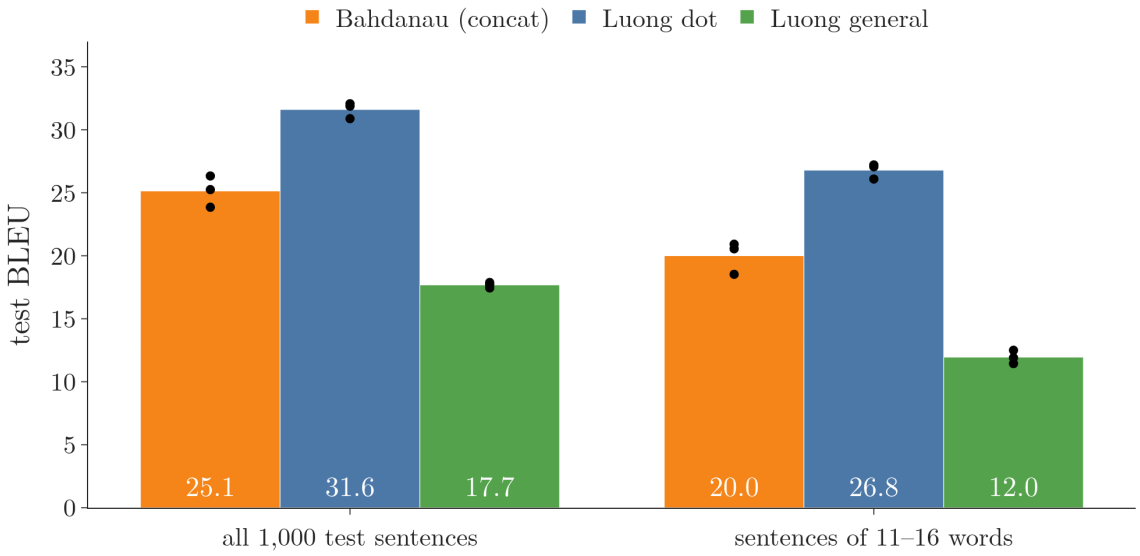


Figure 5: Test BLEU of the three models, on all test sentences and on the long ones. Bars: mean of 3 runs; dots: the single runs

In Figure 5 the runs of each model sit close together, while the gaps between the models are several BLEU points: the ranking dot, then Bahdanau, then general holds for every run and both sentence groups.

The Bahdanau numbers differ slightly from those of attention against no attention by sentence

length (25.7) because GPU training is not exactly repeatable. The differences between the models are much larger than that.

Parameters. Bahdanau's model has more because its output layer reads $[s_i; c_i]$ (512 numbers) and its LSTM reads $[y_{i-1}; c_i]$; Luong's output layer reads the 256-number $\tilde{h}_i$. The scores themselves are a small part: the dot score has no parameters, and general adds one 256×256 matrix (65,536 numbers).

Time. In Bahdanau's model, c_i is an input of LSTM step i, so each step must wait for the attention of that step: the decoder runs as a loop of small operations. In Luong's model the LSTM never needs c_i, so during training (with **teacher forcing**, G-1955: feeding the decoder the true previous word, see the forward pass and teacher forcing) the decoder LSTM runs over the whole gold sentence in one call, and the attention for all steps at once is two matrix products. Stack the decoder states as the rows of a matrix S, the encoder states as the rows of H, and the weights as the rows of A; then $SH^\top$ gives all the scores and AH all the contexts. Each epoch took less than half the time.

Quality. Luong's dot attention gave the best translations at all lengths. The general score did worst; Luong et al. (2015, Table 4) also found dot better than general for global attention (BLEU 18.6 against 17.3 on English–German).

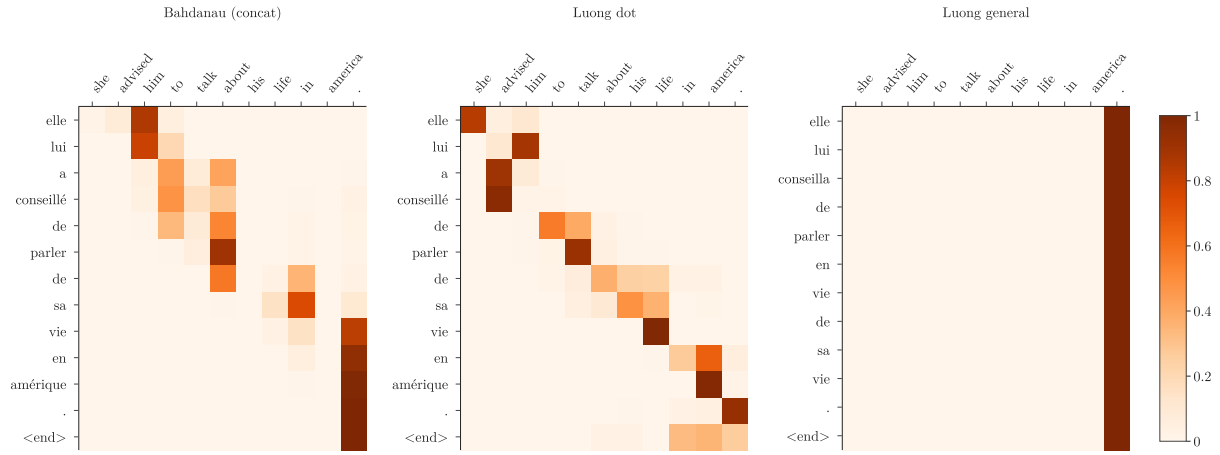


Figure 6: Attention weights of the three trained models on the same real test sentence. Bahdanau and Luong dot both translate it exactly; Luong general puts all its weight on the final full stop

Figure 6 shows why the general model did badly. On the test sentence "she advised him to talk about his life in america .", the dot model's weights form a clean diagonal: "elle" looks at "she", "lui" at "him", "conseillé" at "advised", "parler" at "talk", "vie" at "life", "amérique" at "america". Bahdanau's model also forms a band, but it often sits to the right of the matching word ("parler" looks at "about"). **Attention weights** (G-224) need not match a word alignment, and the same off-by-one attention has been reported in a trained Bahdanau-type system (see seeing the alignment). The general model puts a weight of 1 on

the final “.” at every step. Its context vector is then the same at every step: the state of the last word, whose forward half has read the whole sentence. The model has turned itself back into a plain encoder–decoder with one fixed summary, and its translation of this sentence goes wrong at the end (“elle lui conseilla de parler en vie de sa vie .”).

7. Summary

	Bahdanau	Luong
Score	additive: small network on $[s_{i-1}; h_j]$	multiplicative: $s_i^\top h_j$ or $s_i^\top W_a h_j$
Decoder state used	previous (s_{i-1})	current (s_i)
Context vector enters	the LSTM step	the output layer, via $\tilde{h}_i$
In the Notebook	BLEU 25.1, 51 s per epoch	dot: BLEU 31.6, 22 s per epoch

- The last row is why Luong dot won in the Notebook: fewer parameters, less than half the time per epoch, and higher BLEU on the same data.
- Both compute weights α_{ij} by a softmax over scores and a context vector $c_i = \sum_j \alpha_{ij} h_j$, so the only real choices are the **score function** (G-1752) and where the context goes.
- Bahdanau scores the previous decoder state with a one-hidden-layer network; the context is an input of the next LSTM step, so each step must wait for its own **attention** (G-226) and the decoder runs as a loop.
- Luong scores the current decoder state with a **dot product** (G-634) (or a bilinear form), because a dot product is large when two vectors point the same way and so already measures similarity; it joins the context to the state after the LSTM step, so the LSTM never waits for the attention.
- The dot product needs no parameters and, without **input feeding** (G-949), lets attention for all decoder steps be computed with matrix products, which is why each epoch took less than half the time.
- A learned score can also go wrong: the general model put all its weight on the final full stop, so it fell back to one fixed summary (BLEU 17.7).
- The dot-product score is the one the transformer's self-attention builds on, so this is the difference to remember: a learned network score against a plain dot-product score, and context fed in before the step against joined after it.

8. Sources

Built from

- CampusX, "Bahdanau Attention Vs Luong Attention", YouTube, https://www.youtube.com/watch?v=

Other references

- Bahdanau, D., Cho, K. and Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR 2015*. arXiv:1409.0473. Section 3.1 (eq. 4–6); appendix A.1.2 (the alignment model $v_a^\top \tanh(W_a s_{i-1} + U_a h_j)$, pre-computing $U_a h_j$).
- Luong, M.-T., Pham, H. and Manning, C. D. (2015). Effective Approaches to Attention-based Neural Machine Translation. *EMNLP 2015*. arXiv:1508.04025. Section 3 (eq. 5–8: attentional hidden state, the dot, general and concat scores); 3.1 (comparison with Bahdanau et al.); 3.3 (input feeding); Table 4 and section 5.3 (choice of score function).

- Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS 2017*. arXiv:1706.03762. Section 3.2.1 (additive and dot-product (multiplicative) attention).
- Jurafsky, D. and Martin, J. H. *Speech and Language Processing*, 3rd ed. draft (19 August 2026), web.stanford.edu/~jurafsky/slp3. Chapter 14, §14.8 (dot-product attention; the bilinear score allows states of different sizes). Cited as SLP3.
- Tatoeba project and manythings.org/anki: the English–French sentence pairs, distributed as `fra-eng.zip` with the Keras examples.

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Score function (G-1752)	In attention, the rule that gives the raw score e_{ij} of how well encoder state j matches decoder step i; a softmax turns these scores into attention weights, and Bahdanau and Luong attention differ in this rule.
Bahdanau attention (additive) (G-255)	Attention whose alignment score comes from a small network with one hidden layer, fed the previous decoder state and one encoder state, $v^\top \tanh(W_a s_{i-1} + U_a h_j)$; the score decides how much that encoder state counts. Called additive because the two terms are added inside the tanh.
Luong attention (multiplicative, dot) (G-1137)	Attention between a decoder state and the encoder states, scored by multiplying them: $s_i^\top h_j$ (dot) or $s_i^\top W_a h_j$.
Attentional hidden state $\tilde{h}_i$ (G-227)	In Luong attention, the vector the output layer reads to predict the next word: the decoder state and the context vector joined and passed through a tanh layer, $\tilde{h}_i = \tanh(W_c[c_i; s_i])$.
Input feeding (G-949)	Joining the previous step's attention output $\tilde{h}_{i-1}$ to the next decoder input, so the model remembers its past alignment choices; an option in Luong's attention.

Term	Meaning
Concat score (G-435)	A score for how useful an encoder state is now: join the decoder state and the encoder state into one vector and pass it through a tanh layer and one output unit to get one number, $v_a^\top \tanh(W_a[s_i; h_j])$. It is Luong's name for the additive (Bahdanau) attention score.
Dot score (G-635)	Luong's simplest attention score: the dot product of the decoder state and an encoder state, $s_i^\top h_j$, large when the two point the same way. It needs no learned parameters, but both states must have the same size.
General score (G-837)	A way to score how well a decoder state s_i matches an encoder state h_j through a learned matrix, $s_i^\top W_a h_j$ (Luong's general score). It lets the two states have different sizes and lets the model learn which directions of similarity matter.
Dot product (G-634)	Multiplying two vectors entry by entry and adding the products, $u^\mathsf{T} x$, which gives one number. It is large when the vectors point the same way, so it measures similarity and gives projections.
Attention (G-226)	A mechanism that lets each output step weigh all input positions and focus on the useful ones.
Context vector c_i (G-460)	The summary of the input made fresh for each decoder step i under attention: a weighted sum of all the encoder's hidden states, where the weights decide how much each input word counts for the word being written now.
Attention weight α_{ij} (G-224)	How much encoder state j counts at decoder step i; the weights of one step are non-negative and sum to 1.
Softmax function (G-1830)	A function that turns a list of scores into probabilities that add up to 1: raise e to each score and divide by the total, $e^{z_k} / \sum_j e^{z_j}$; it gives one probability per class in multi-class models.
Alignment score e_{ij} (G-190)	How strongly encoder state j fits decoder step i, as a raw score before the softmax turns the scores into attention weights.

Term	Meaning
Forward propagation (G-797)	Passing one row of inputs through the network, layer by layer, to get the prediction.
Feed-forward network (G-775)	A network in which information moves only from the first layer to the last.
Alignment model (G-189)	The small feed-forward network that scores how well encoder state h_j fits the previous decoder state s_{i-1}, giving the alignment score e_{ij}.
BLEU score (G-315)	A measure of translation quality: how many word sequences of a translation match a human reference.
Teacher forcing (G-1955)	Feeding the correct (gold) previous token to the decoder during training instead of its own prediction, so every step learns from a correct history and training goes faster.